ASCENDION

Capstone: TerraFlow T-300

OpenAI Overview and AI Awareness for Firmware Teams: capstone

By the end of this module, you will be able to

1. **Diagnose a field problem with AI**
   Logs and reports triaged, with evidence checked in code
2. **Review and test firmware with AI**
   Legacy driver, datasheet header, unit tests from the spec
3. **Prove every result**
   Datasheet, requirement, tests, sanitizers and a scanner
4. **Decide what is safe**
   AI output reviewed, team rules written, a first use case chosen

The field problem

False dry-run stops

P-401 while water is flowing; restarts, then stops again

Wrong pressure

The app shows 2.2 bar at 1.5 bar, or a value that never changes

Resets and locks

TF-41077 resets on its watchdog; a pump locked after its third stop

AI-written code

Someone used AI to rewrite the dry-run handling

An email ready to send

An AI-drafted customer update, about to go to every customer

The client wants three answers by the end of the day: what is wrong, what is safe to send, and where AI should help next.

The product: TerraFlow T-300

PX-12 pressure sensor	I2C at 0x28, read every 500 ms; 14-bit output, 1638 to 14745 counts for 0 to 10 bar
Motor duty from pressure	pump_spec.md: 0% at 3.00 bar, 30% to 100% below, rounded half up
Dry-run and sensor protection	REQ-310: stop after 10 low readings, P-401; P-402 on a bad sensor; lock with P-403
LoRa and the app	Remote status; out of scope for this capstone

Two sources of truth: pump_spec.md for the duty, REQ-310 for protection. The datasheet for the sensor.

The client’s rules for this engagement

Approved tools only	The company Foundry project and Copilot Business in the repository; no personal AI accounts
Never in a prompt or a deliverable	Keys, tokens, connection strings, customer names or contact details
Every AI output is a draft	Values, code and claims count only once checked against a source
Safety functions stay on	Dry-run and sensor-fault protection are never disabled on AI advice

Breaking a rule costs points: up to 10 for a leak, 5 for advising to disable protection.

Your kit

Folder	What is in it
docs/	pump_spec.md, REQ-310, PX-12 datasheet page, release notes, FAQ
data/	Boot log of TF-41077 and ten field reports
src/ and tests/	Legacy driver, pump control (planted defects), plain C unit tests with fakes
ai_outputs/	C1 dry-run patch and C2 customer update, to review
scripts/ and verify/	t1 to t6, one per task, all with --mock; run_checks.py
deliverables/	Team report template and readout outline: save everything here

Eight tasks, five modules

Task	What you do	Modules
1	Settings, grounding, token budget	1, 2
2	Triage reports and the boot log	2, 3
3	Review the legacy driver	3, 4
4	Datasheet to a checked header	4

Task	What you do	Modules
5	Unit tests from the spec; fix	4
6	Knowledge assistant, cited	4
7	Review C1 and C2; team rules	5
8	Canvas and readout	4, 5

Every task has inputs, steps and a “done when” list in the problem statement document.

Part A: understand the problem (45 minutes)

Task 1

Settings

Duty at 2.10 bar at two temperatures; a trap, grounded; tokens for 200 calls

→

Task 2

Triage

Boot log with the template; reports as JSON; quotes checked word for word

→

Task 3, first half

Prompt

Your five-part review prompt, with datasheet and REQ-310

→

Output

Evidence

Three patterns, each tied to reports, log lines and a criterion

Know the answer before the model does: work out 2.10 bar from the spec yourself first.

Part B: check and fix (65 minutes)

Task 3

Finish the defect table; link each report to a defect

Task 4

Datasheet to header; checks on CONFIG and the transfer function

Task 5

Expected values from the spec first; tests; find and fix

Task 6

Five questions with citations; two of your own

⚠ **Some defects need the datasheet, some need REQ-310.** A prompt with the code alone will miss them.

Part C: decide (40 minutes)

Task 7: review and rules

C1: read against REQ-310, then run the checks

C2: may it be sent? Rewrite it or stop it

One page of team rules: before, during, after

Task 8: canvas and readout

The first TerraFlow task for production

Measured pain, data rule, check, first step

Five slides, five minutes

Before you hand in: python verify/run_checks.py scan must be clean for deliverables/.

Schedule

Time	Part	What happens
0:00 to 0:10	Kick-off	Scenario, rules, teams and roles, setup check
0:10 to 0:55	A: understand	Tasks 1, 2 and the first half of 3
0:55 to 2:00	B: check and fix	Task 3 finished, Tasks 4, 5 and 6
2:00 to 2:40	C: decide	Tasks 7 and 8; deliverables saved; scan clean
2:40 to 3:20	Readouts	Five minutes per team, then questions
3:20 to 3:30	Wrap-up	Scores, the defects nobody found, what to take back

Team roles

Lead and integrator

Tasks 1, 2 and 8; the report and the readout; watches the clock

Code reviewer

Task 3 and the C1 review in Task 7

Test engineer

Tasks 4 and 5; expected values from the spec

Safety and data reviewer

Tasks 6 and 7; the rules; the final scan

Three people? The lead also takes Task 6, and everyone reviews C2 together.

What you hand in

IN deliverables/

- **team_report.md**
  the template, filled in
- **review_prompt.md and defect table**
  Task 3
- **px12_regs.h**
  Task 4, corrected
- **test_pump_control.c and your fix**
  Task 5

ALSO

- **C2_rewritten.md**
  or a reasoned decision not to send
- **canvas.md**
  Task 8
- **Five readout slides**
  any format
- **A clean scan**
  run_checks.py scan

Scoring: 100 points

Task	Pts	Full marks when
1. Settings	10	Value from the spec; grounding shown
2. Triage	10	Checks pass; no invented causes
3. Driver review	20	All three defect groups; lines checked
4. Header	10	Every value matches the PDF

Task	Pts	Full marks when
5. Tests	15	Values from the spec; all pass after fix
6. Assistant	10	Cited, or “Not in the docs.”
7. AI output	15	All mistakes; C2 safe or stopped
8. Canvas	10	Frequent, checkable first use case

⚠ **Deductions:** up to 10 for a leak, 2 per unsourced AI claim, 5 for advising to disable protection.

How strong teams work

Split the work in the first five minutes	Roles, owners per task, a time for the first merge
Source first	Find every value in the spec, REQ-310, the datasheet or the log, or it does not count
Scripts check shape; people check meaning	Lines, quotes, resets and scans by code; causes and safety by you
Keep a list of what the AI got wrong	It earns points in Tasks 3, 6 and 7, and makes the best readout slide

The goal is not to use AI everywhere. It is to use it where you can prove the result.

The readout: five slides, five minutes

1 · Problem

Evidence

What the reports and the log show

→

2 · Findings

Checked

Top defects, and how each was confirmed

→

3 · Checks

Caught

One catch each from code, tests and review

→

4 and 5 · Decide

Safe and next

C1, C2, your rules; your first use case

Questions we will ask: how do you know, what did the AI get wrong, and what would you not let it do?

ASCENDION

Thank you

Questions and discussion